

Software Engineering

Lecture 7b: Black-Box Testing Techniques

Prof. Nien-Lin Hsueh

Department of Information Engineering and Computer Science
Feng Chia University

Focus Questions

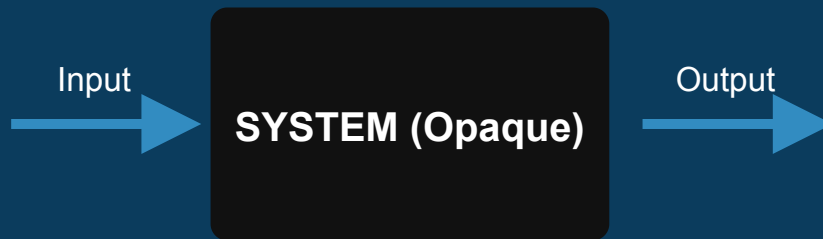
- What is **Black-Box (Specification-Based) Testing**?
- What are the 4 **Boundary Value Testing Formulas** ($4n+1$, $6n+1$, 5^n , 7^n) and Single Fault Assumption?
- How do **Weak Coverage (Weak EP)** and **Strong Coverage (Strong EP)** differ?
- How do we design test suites for **Triangle Classification, Exam Scores, FCU Swimming Pool Fee, Binary Search, and nextDate()**?
- How does **All-Pairs (Pairwise) Testing** solve combinatorial explosion?
- How do **Decision Table Testing** and **State Transition Testing** model system logic?

Black-Box vs. White-Box Testing

- **Black-Box Testing (Functional / Specification-Based):**
 - **Definition:** A testing method where the internal code structure, implementation details, and source code are **completely hidden** from the tester.
 - **Focus:** Tests system behavior against functional specifications by sending inputs and validating expected outputs.
- **White-Box Testing (Structural / Glass-Box Testing):**
 - **Definition:** A testing technique where internal code structure and control flow are **fully visible**.

Black-Box Testing

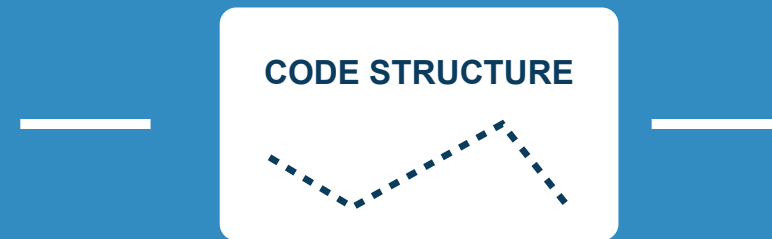
(Specification-Based)



- No internal code knowledge
- Tests functional requirements & specs
- Equivalence partitioning & boundary values

White-Box Testing

(Structural / Glass-Box)



- Full visibility into source code
- Tests internal logic, branches & loops
- Statement, branch & path coverage

5 Major Black-Box Testing Methods

5 Key Black-Box Testing Techniques

1. Equivalence Partitioning

Divides input domain into valid & invalid equivalence classes.

2. Boundary Value Analysis

Tests values at partition edges (min-1, min, max, max+1).

3. Decision Table Testing

Maps combinations of complex logical conditions to actions.

4. State Transition Testing

Verifies system responses across discrete finite state transitions.

5. Use Case / Scenario

Derives test cases directly from end-to-end user workflows.

Boundary Value Testing Taxonomy

- **Single Fault Assumption (單一錯誤假設):**
 - Assumes a system failure is caused by a defect in a *single variable* rather than simultaneous defects across multiple variables.
- **Independent vs. Non-Independent (Worst-Case):**
 - **Independent:** Variables do not interact; test boundary of 1 variable holding others at `norm`.
 - **Non-Independent (Worst-Case):** Variables interact in logic statements (e.g. `if (exam <= 60 && hw <= 60)`). Requires Cartesian cross-product of boundary values.

The 4 Boundary Testing Formulas

Black-Box Boundary Value Testing Taxonomy

Testing Type / Assumption	Normal Boundary (min, max)	Robust Boundary (min-, max+)
Independent Testing (Single Fault Assumption)	$4n + 1$ min, min+, norm, max-, max	$6n + 1$ Adds out-of-bounds (min-, max+)
Non-Independent (Worst-Case / Cartesian Product)	5^n Combinatorial cross-product ($5^3 = 125$)	7^n Robust cross-product ($7^3 = 343$)

Boundary Case Study: Triangle Classification

“**Requirement:** Inputs $a, b, c \in [1, 200]$. Output: *Equilateral*, *Isosceles*, *Scalene*, *Non-Triangle*. ”

- **Independent Normal Boundary ($4n+1 = 13$ cases):**
 - Hold b, c at `norm=100`, test $a \in \{1, 2, 199, 200\}$. Repeat for b and c , plus 1 `norm` case.
- **The Diversity Deficit & Solution:**
 - Independent testing (13 cases) only produces *Isosceles* and *Equilateral* triangles, missing *Scalene* triangles!
 - **Solutions:** Use dynamic random `norm` values or apply **Output-Guided Partitioning**.
- **Non-Independent Worst-Case:** $5^3 = 125$ test cases ($5 \times 5 \times 5$).

Equivalence Partitioning: Weak vs. Strong Coverage

- **Equivalence Partitioning Concept:**
 - Divides input/output space into classes where all values are processed identically.
- **Weak Coverage (Weak EP):**
 - Based on Single Fault Assumption. Every partition of every variable is tested **at least once**.
 - Total test cases = $\max(\text{Partitions of } v_1, \dots, v_n)$.
- **Strong Coverage (Strong EP):**
 - For non-independent interacting variables. Tests the **Cartesian product** of all partitions across all variables ($P_1 \times P_2 \times \dots \times P_n$).

EP Case Study 1: Single-Variable Domain (Exam Scores)

“**Requirement:** An online grading system accepts exam scores between **0** and **100** inclusive.”

- **Partition 1: Invalid Low (Score < 0)**
 - *Representative Test Value: -15 → Expected Output: Error ("Score cannot be negative").*
- **Partition 2: Valid Range ($0 \leq \text{Score} \leq 100$)**
 - *Representative Test Value: 75 → Expected Output: Valid grade recorded.*
- **Partition 3: Invalid High (Score > 100)**
 - *Representative Test Value: 135 → Expected Output: Error ("Score exceeds maximum 100").*

EP Case Study 2: Multi-Variable FCU Swimming Pool Fee

“*Requirement: Public pool ticket price depends on **Age**, **Time Slot**, and **Membership**.*”

- **Input Partitions:**

- **Age:** Child (< 12), Adult (12–64), Senior (≥ 65), Invalid (< 0).
- **Time Slot:** Off-Peak (Morning) vs. Peak (Evening).
- **Membership:** Member (Discount) vs. Non-Member (Full Price).

 Swimming Pool EP Matrix

EP Case Study 3: Binary Search Partition Plan

“**Spec:** `Search(Key: int, A: Array) -> (Found: bool, Index: int)`”

- **Input Array Partitions:**

- Empty array (a^0 : empty), Single element (a^1 : 1 element), Multiple elements (a^* : multiple elements).

- **Output & Position Partitions:**

- **Found (f^t):** Key at first position (c^1), middle position (c^m), or last position (c^l).
- **Not Found (f^f):** Key not present in array.
- **Invalid Type (k^l):** Error handling for corrupted input.

- **Weak Coverage Goal:** Derive 8 structured test cases ($R_1 - R_8$) covering all input and output partitions.

EP Case Study 3: Binary Search Test Suite

“Concrete Test Suite (R1 – R8) covering all Equivalence Partitions:”

Binary Search EP Weak Coverage Test Suite (8 Test Cases R1 - R8)

Case	Partition Rule Description	Key	Input Array (A)	Expected Output (Found, Index)
R1	1 element, Found	5	[5]	(Found=True, Index=0)
R2	1 element, Not Found	5	[8]	(Found=False, Index=-1)
R3	Multi elements, First Pos (c1)	5	[5, 12, 20]	(Found=True, Index=0)
R4	Multi elements, Mid Pos (cm)	12	[5, 12, 20]	(Found=True, Index=1)
R5	Multi elements, Last Pos (cl)	20	[5, 12, 20]	(Found=True, Index=2)
R6	Multi elements, Not Found	99	[5, 12, 20]	(Found=False, Index=-1)
R7	0 elements (Empty Array)	5	[]	(Found=False, Index=-1)
R8	Invalid Input / Null Key	null	[5, 12, 20]	ERROR: Invalid Input

Boundary & EP Case Study: nextDate Plan

“**Spec:** `nextDate(month: int, day: int, year: int) -> String` (Date in 1800–2048)

”

nextDate Case Study

- **Key Boundary Test Points:**

- **Month End:** `2024-01-31` → `2024-02-01` | **Year End:** `2024-12-31` → `2025-01-01`
- **30-Day Boundary:** `2024-04-30` → `2024-05-01` (Day `31` in April is Invalid).
- **Leap Year Boundary:** `2024-02-28` → `2024-02-29` vs. `2023-02-28` → `2023-03-01`.

Boundary & EP Case Study: nextDate Test Suite

“Concrete Test Suite (TC1 – TC8) covering all Boundary & Equivalence Classes:”

nextDate(month, day, year) Concrete Test Suite (TC1 - TC8)

Case	Boundary / Rule Category	Input Date (m, d, y)	Expected Result / Output
TC1	Normal Day Increment	month=3, day=15, year=2024	2024-03-16
TC2	31-Day Month End	month=1, day=31, year=2024	2024-02-01 (Month Rollover)
TC3	30-Day Month End	month=4, day=30, year=2024	2024-05-01 (April Rollover)
TC4	30-Day Invalid Day	month=4, day=31, year=2024	ERROR: Apr 31 Invalid
TC5	Leap Year Feb 28	month=2, day=28, year=2024	2024-02-29 (Leap Day)
TC6	Non-Leap Year Feb 28	month=2, day=28, year=2023	2023-03-01 (March 1)
TC7	End of Year Rollover	month=12, day=31, year=2024	2025-01-01 (New Year)
TC8	Out-of-Bounds Month	month=13, day=15, year=2024	ERROR: Invalid Month

All-Pairs (Pairwise) Testing

- **The Combinatorial Explosion Problem:**

- Testing all combinations (Strong EP / Cartesian Product) across k variables requires $O(N^k)$ test cases (e.g. $3^{10} = 59,049$ tests).

- **Core Empirical Principle:**

- *Most software bugs are triggered by interactions between at most 2 parameters (pairs) rather than complex 5-way interactions.*

Combinatorial Explosion vs. All-Pairs (Pairwise) Testing

Full Combinatorial (Cartesian)

Tests all possible combinations across all N variables ($N_1 \times N_2 \times \dots \times N_k$).

Exponential Explosion: $O(N^k)$

e.g. 10 variables w/ 3 values = 59,049 tests!

VS

All-Pairs (Pairwise Testing)

Ensures EVERY PAIR of variables is tested together at least once.

Drastic Reduction: $O(N^2)$

e.g. 59,049 tests reduced to ~15-20 tests!

Black-Box Method 3: Decision Table Testing

- **Concept:**

- A structured tabular approach to model complex business logic containing multiple conditional rules.
- **Structure:** Combines input conditions (True/False) with resulting system actions.

Decision Table Example: E-Commerce Discount Rules

Conditions / Actions	Rule 1	Rule 2	Rule 3	Rule 4
Is VIP Member?	T	T	F	F
Order Total > \$100?	T	F	T	F
Action: Apply Discount	20% Off	10% Off	5% Off	0% (None)

Black-Box Method 4 & 5: State & Use Case Testing

- **Method 4: State Transition Testing:**

- Tests how the system transitions between discrete finite states in response to user events.
- Verifies both **valid transitions** (e.g. *Unauthenticated* → Login → *Authenticated*) and **invalid transitions** (e.g. attempting checkout while unauthenticated).

- **Method 5: Use Case / Scenario Testing:**

- Test cases are derived directly from high-level **Use Cases** or user stories.
- Verifies end-to-end user goals, basic normal flows, alternate paths, and exception flows.

Concept Check Question 1

In Black-Box Test Case Design, why is Boundary Value Analysis (BVA) used alongside Equivalence Partitioning?

- **A.** Because white-box code source files are required for boundary testing.
- **B.** Because software defects occur most frequently at the edges/boundaries of input partitions.
- **C.** Because boundary values eliminate the need for unit testing.
- **D.** Because BVA only tests valid partitions and ignores invalid inputs.



Concept Check Question 1: Answer

In Black-Box Test Case Design, why is Boundary Value Analysis (BVA) used alongside Equivalence Partitioning?

- **Correct Answer: B**
- **Explanation:** Off-by-one errors ($<$ vs $\leq$) mean defects concentrate heavily at boundaries.



Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. testing divides input domains into classes processed identically by the system.
2. Under Single Fault Assumption, Independent Normal Boundary testing requires test cases for n variables.
3. For interacting non-independent variables, worst-case boundary testing requires test cases.
4. testing ensures every pair of input parameters is tested together at least once, avoiding combinatorial explosion.



Recap: Answers & Summary

Here are the completed concepts:

1. **Equivalence partitioning** testing divides input domains into classes processed identically by the system.
2. Under Single Fault Assumption, Independent Normal Boundary testing requires $4n + 1$ test cases for n variables.
3. For interacting non-independent variables, worst-case boundary testing requires 5^n test cases.
4. **All-pairs (pairwise)** testing ensures every pair of input parameters is tested together at least once.



References

- **Prof. Nien-Lin Hsueh Software Testing Course (Ch05 Black-Box Testing)**
 - [IEEE 829 Standard for Software Test Documentation](#)
 - [Pairwise / All-Pairs Testing Tool PICT](#)